CSCI419 – Theory of Computation

Project Report

NFA to DFA Converter

Team Members

[Karim Bassem Joeseph – ID: 231000797]

[Mathew Hany Samir – ID: 231000574]

[Ali Essam Ali – ID: 231001040]

[Hagar gamal – ID: 19105329]

1. Project Description

This project implements the Subset Construction Algorithm to convert a Nondeterministic Finite Automaton (NFA) with epsilon (ε) transitions into an equivalent Deterministic Finite Automaton (DFA). NFAs are powerful theoretical models, but they allow multiple possible transitions for a single input symbol and can transition without consuming any character (ε-transitions). While this makes NFAs convenient for design, computers require a deterministic machine where each state has exactly one transition per symbol. This engine bridges that gap: it reads an NFA definition, applies the well-known subset construction procedure, and outputs the fully computed DFA transition table.

The project is a zero-dependency Python engine with no external libraries used. Every algorithm — graph traversal, closure computation, state naming, and table formatting — is implemented from scratch. NFAs are supplied via JSON files, making it easy to define and swap test cases. The final DFA is printed as a formatted transition table to the terminal, clearly marking the start state and all accepting states.

2. Input Format

The NFA is defined in a JSON file with the following structure:

```
{
  "states":             ["q0", "q1", "q2", "q3"],
  "alphabet":           ["a", "b"],
  "start_state":        "q0",
  "accepting_states":   ["q3"],
  "transitions": {
    "q0": { "a": ["q0", "q1"], "b": ["q0"] },
    "q1": { "b": ["q2"] },
    "q2": { "b": ["q3"] }
  },
  "epsilon_transitions": {
    "q0": ["q1"]
  }
}
```

Key fields:

- states — list of all NFA state names.
- alphabet — list of input symbols (e.g., "a", "b").
- start_state — the initial state of the NFA.

- accepting_states — list of accepting / final states.
- transitions — maps each state to its symbol-based transitions (each symbol maps to a list of target states, since the NFA is nondeterministic).
- epsilon_transitions — maps states to states reachable via ε (no input consumed).

3. Output Format

The program prints a formatted DFA transition table to the terminal. Each DFA state is assigned a single capital letter (A, B, C, ...) via the Subset Construction naming scheme. The start state is prefixed with → and accepting states are prefixed with *.

Sample output for nfa_complex.json:

```
--- DFA TRANSITIONS TABLE ---
State      | a     | b
-------------------------------
->A        | A     | B
B          | A     | C
*C         | A     | C
-------------------------------
```

The table above as a formatted transition table:

State	a	b
→ A (start)	A	B
B	A	C
* C (accept)	A	C

Notation used in the output:

- → marks the start state (DFA state A corresponds to ε-closure({q0})).
- * marks an accepting state (any DFA state containing an NFA accepting state).
- - (dash) means no transition exists for that symbol from that state.

4. Inside Mechanism

The core of the engine follows the classical Subset Construction (Powerset Construction) algorithm. Below is a step-by-step description of how it works internally.

4.1 epsilon_closure(state)

Computes all NFA states reachable from a given state using only ε-transitions (without consuming any input). It uses a Depth-First Search (DFS) with a stack and a visited set to avoid infinite loops. The result is a set of states that the machine is implicitly "in" whenever it enters the given state.

4.2 move(states, symbol)

Given a set of NFA states and an input symbol, this function returns all NFA states reachable by: (1) following the symbol-based transitions from any state in the set, then (2) computing the ε-closure of each resulting state. This captures all immediate and epsilon-reachable destinations in one step.

4.3 nfa_to_dfa(start_node, alphabet, nfa_accepting_states)

This is the main conversion engine. It runs the Subset Construction algorithm:

- Compute ε-closure of the NFA start state → this becomes DFA state A.
- Use a queue of unprocessed DFA states. Start with state A.
- For each DFA state (a set of NFA states) and each symbol in the alphabet, call move() to find the next set of NFA states.
- If the resulting set is new, assign it the next letter name and add it to the queue.
- Record the transition (DFA_state, symbol) → next_DFA_state.
- A DFA state is accepting if it contains any NFA accepting state.
- Repeat until the queue is empty.

4.4 read_nfa_from_json(filename)

Reads the NFA JSON file and constructs a linked graph of State objects. It does this in two passes: first creating all State objects, then resolving string names into actual object references to handle forward references correctly.

4.5 print_dfa(...)

Formats and prints the DFA transition table to the terminal. It dynamically sizes columns based on the alphabet, prefixes the start state with → and accepting states with *, and fills in - where no transition exists.

5. Programming Language, Tools & Libraries

- Programming Language: Python
- External Libraries: None (zero-dependency project — all algorithms implemented from scratch).
- Standard Libraries used: json (for reading NFA definition files).
- Input Format: JSON files (nfa.json, nfa_complex.json).
- IDE / Environment: Compatible with any Python 3 environment; tested via terminal with python3 main.py and also works perfectly with VS code.

6. Project Structure

File	Description
`main.py`	Entry point. Reads the JSON, runs the conversion, prints the DFA table.
`classes.py`	Defines the State class (node of the NFA graph) with transitions and epsilon_transitions to be easier to track the state.
`utils.py`	All logic: epsilon_closure, move, read_nfa_from_json, nfa_to_dfa, get_dfa_state_name, print_dfa.
`nfa.json`	Simple 3-state NFA input file.
`nfa_complex.json`	Complex 4-state NFA input file (used by default).

7. How to Run

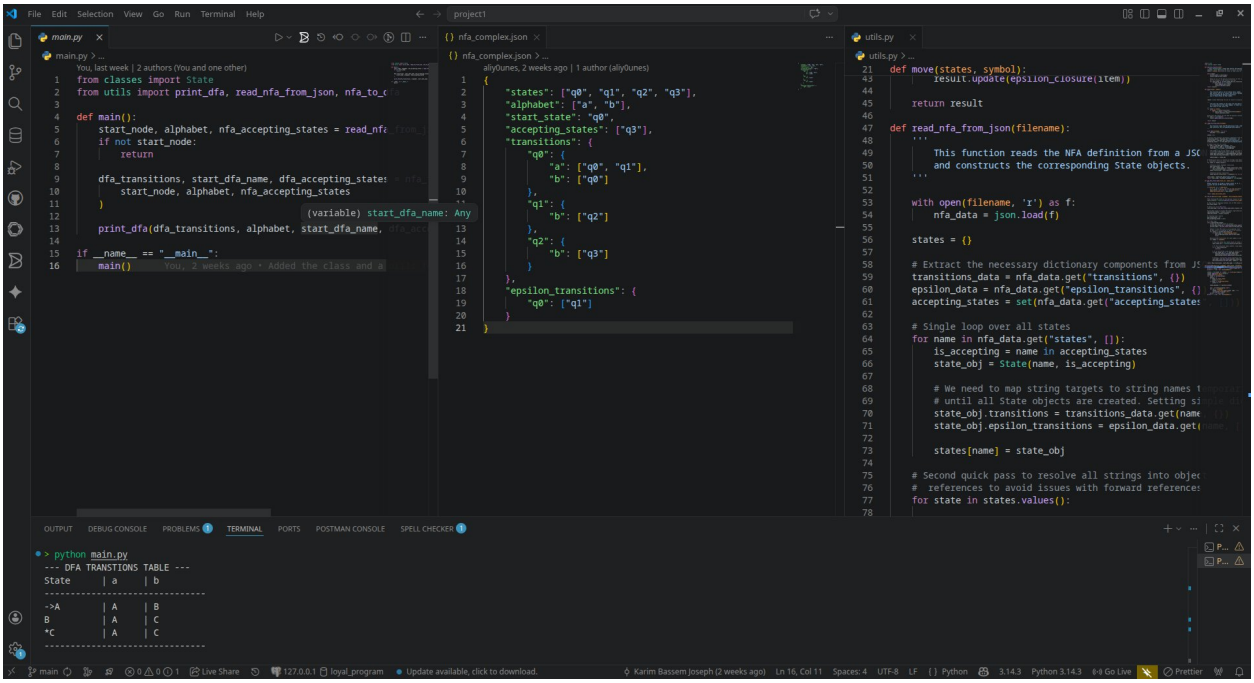
Ensure Python 3 is installed, then run from the project directory:

```
python3 main.py
```

By default, the program loads nfa_complex.json. To switch to the simple NFA, open main.py and change the filename argument in read_nfa_from_json("nfa_complex.json") to "nfa.json".

8. Project Output

The following shows the terminal output produced when running the project with nfa_complex.json:



Terminal output showing the computed DFA transition table from nfa_complex.json with the main.py and the nfa_complex.json and the utils.py file from VS code.